

Clase 09 (05/19/2026)

1. Gazebo

- Importancia de los entornos de simulación:
 - Esfuerzo, tiempo y costo de probar hardware real
 - Prueba de algoritmos con acciones que pueden presentar peligros y riesgo el código no probado
 - Aislar variables
- Gazebo: gratis, open-source, de 'Open Robotics'
- Entorno virtual y robot simulado: Sensores, actuadores y feedback
- Cambio de `joint_state_publisher_gui` (valores artificiales) por estados simulados desde Gazebo

1.1. Archivos SDF

- *SDF*: Simulation Description Format. Permite describir un mundo además de modelos y robots que se encuentran en el mismo.
- Se puede convertir *URDF* a *SDF*. Requisitos: Nombramiento y propiedades inerciales y físicas de cada link:

```
<link name="[nombre_link]">
  <!-- .. -->
  <inertial>
    <origin xyz="[traslacion]" rpy="[rotacion]" />
    <mass value="[masa_kg]" />
    <inertia ixx="[I_xx]" ixy="[I_xy]" ixz="[I_xz]"
            iyy="[I_yy]" iyz="[I_yz]"
            izz="[I_zz]" />
  </inertial>
  <!-- .. -->
</link>
```

- `inertial_macros.xacro`

- Prisma

```
<xacro:inertial_box mass="[masa_kg]" x="[largo]" y="[ancho]" z="[alto]">
  <origin xyz="[traslacion]" rpy="[rotacion]" />
</xacro:inertial_box>
```

- Cilindro

```
<xacro:inertial_cylinder mass="[masa_kg]" radius="[radio]" length="[ancho]">
  <origin xyz="[traslacion]" rpy="[rotacion]" />
</xacro:inertial_cylinder>
```

- Para cuando sea necesario una inercia despreciable

```
<dummy_inertial />
```

- Cargar URDF en gazebo desde archivo *launch*

```
Node(
  package="ros_gz_sim",
  executable="create",
  arguments=[
    "-entity", "<nombre_robot>",
    "-topic", "robot_description",
  ],
  output="screen",
)
```

2. ros2_control

- Conjunto de paquetes para desarrollar controladores genéricos para todo tipo de robots
- ros2_control + Gazebo = gz_ros2_control
- Definir un hardware simulado con <ros2_control> y añadir el plugin gz_ros2_control

```
<ros2_control name="GazeboSystem" type="system">
  <hardware>
    <plugin>gz_ros2_control/GazeboSimSystem</plugin>
  </hardware>
  <!-- Definición de las interfaces -->
</ros2_control>
<!-- .. -->
<gazebo>
  <plugin filename="gz_ros2_control-system"
    name="gz_ros2_control::GazeboSimROS2ControlPlugin">
    <parameters>
      $(find [nombre_paquete])/config/[nombre_archivo].yaml
    </parameters>
  </plugin>
</gazebo>
```

- Para cada junta <joint> definir las interfaces: state_ o command_ interface

```
<ros2_control name="GazeboSystem" type="system">
  <!-- .. -->
  <joint name="[nombre_junta]">
    <!-- Interfaz de posición -->
    <[state|command]_interface name="position" />
    <!-- Interfaz de velocidad -->
    <[state|command]_interface name="velocity" />
    <!-- Interfaz de esfuerzo -->
    <[state|command]_interface name="effort" />
  </joint>
  <!-- .. -->
</ros2_control>
```

- Definir controladores para `ros2_control` en archivo *YAML*
 - `joint_state_broadcaster/JointStateBroadcaster`
 - `velocity_controllers/JointGroupVelocityController`

```

controller_manager:
  ros__parameters:
    update_rate: 30          # Velocidad de actualización de los controladores
    use_sim_time: true       # Utilizar la simulación de Gazebo

    <nombre_controlador>
      type: <tipo_controlador>

    # ..

```
- Configurar controlador `velocity_controllers/JointGroupVelocityController`

```

<nombre_controlador>:
  ros__parameters:
    joints:
      - <nombre_junta>
    # ..

```
- Configurar controlador `joint_state_broadcaster/JointStateBroadcaster`

```

<nombre_controlador>:
  ros__parameters:

    # Listado de juntas a publicar (si no se define se publican todas)
    joints: ["<nombre_junta1>", "<nombre_junta2>", "<..>" ]

    # Marco de referencia (por defecto es 'base_link')
    frame_id: <nombre_marco_referencia>

```
- Cargar controladores desde archivo *launch*:

```

ExecuteProcess(
  cmd=['ros2', 'control', 'load_controller',
      '--set-state', 'active',
      '<nombre_controlador>'],
  output='screen'
)

```